

Encoding multisets of vectors that carry units

A unit-honest method by factorisation

June 25, 2026

Abstract

We are given n vectors with *no preferred order* — a *multiset* — whose entries are physical quantities carrying *units* (a mass, a length, a density, ...). We want to summarise the multiset by a single tuple of numbers that (i) ignores the order of the vectors, (ii) loses no information, (iii) varies continuously, and — the demand special to this note — (iv) is built without ever *comparing or adding two quantities of different units*, since such an operation is physically meaningless and changes wildly when one switches metres for feet. We describe a method that meets all four. Its engine is “encode by factorise”: turn each vector into a polynomial, the whole multiset into a single *product* of such polynomials, and recover the multiset by factorising it back. The unit-honesty is bought cheaply, by carrying each entry’s dimension on its own bookkeeping variable — *one variable per base unit* (mass, length, time), not one per derived unit. The consequence is the headline: the size of the encoding is governed by the small, fixed number of *base* units, not by however many exotic derived units (force, energy, density, ...) happen to appear.

Contents

1	The problem	2
1.1	Multisets of vectors, and what we usually ask of an encoding	2
1.2	The new ingredient: the entries carry units	2
1.3	The demand, and why the obvious methods fail it	3
1.4	The plan	3
2	The engine: encode by factorise	3
2.1	One vector becomes a polynomial	3
2.2	The multiset becomes a product	4
2.3	Reading it back: factorisation	4
2.4	A worked example	4
3	Putting the units back: where the engine sins	5
3.1	The hidden illegal addition	5
3.2	The clumsy cure, and why it is too expensive	5
4	The fix: carry dimensions on base variables	6
4.1	Base accessory variables, and the homogenising monomial	6
4.2	The unit-honest per-vector encoder	6
4.3	The multiset encoder	7
4.4	A worked example with real units	7
5	Why it works	8

6	What it costs	8
6.1	Counting honestly: not every box is filled	8
6.2	Sanity check: uniform units cost nothing extra	9
6.3	The dial	9
7	Summary	10
A	Implementation	10

1 The problem

1.1 Multisets of vectors, and what we usually ask of an encoding

A *multiset* is a collection in which repeats are allowed and counted, but order is ignored: $\{3, 3, 5\}$ and $\{5, 3, 3\}$ are the same multiset, while $\{3, 5\}$ and $\{3, 3, 5\}$ differ. We are interested in multisets of n vectors, each with k entries; we write a vector as a column $\mathbf{v} = (v_1, \dots, v_k)$ and a whole multiset as

$$\{\mathbf{v}^1, \mathbf{v}^2, \dots, \mathbf{v}^n\}, \quad \mathbf{v}^j = (v_1^j, \dots, v_k^j).$$

(Superscripts name the vectors; subscripts name the entries within a vector.) An *encoding* is a map f sending the multiset to a fixed-length tuple of numbers $f \in \mathbb{R}^N$. The three properties usually demanded are:

- (P1) Permutation-invariance:** reordering the n vectors does not change f (it depends on the multiset, not on how we listed it).
- (P2) Injectivity:** different multisets give different f — no information is lost. A map with (P1) and (P2) is an *embedding*.
- (P3) Continuity:** nudging the entries a little moves f a little.

1.2 The new ingredient: the entries carry units

So far the entries were just numbers. But in physics each entry is a *dimensional quantity* — it carries a *unit*. The first entry of every vector might be a mass, the second a length, the third a density. Crucially, the units of *different* entries may be *incommensurable*: a mass and a length cannot be compared or added, and “is this mass bigger than that length?” is not a question with an answer.

Definition 1.1 (units, base and derived). A small fixed set of *base units* is chosen once — in mechanics the usual choice is mass M , length L , time T , so there are $B = 3$ of them. Every other unit is a *derived unit*: a product of integer powers of the base units. We record a derived unit by its *exponent vector* $\mathbf{d} \in \mathbb{Z}^B$. For example, with base order (M, L, T) ,

$$\text{velocity } L/T \leftrightarrow (0, 1, -1), \quad \text{force } ML/T^2 \leftrightarrow (1, 1, -2), \quad \text{density } M/L^3 \leftrightarrow (1, -3, 0).$$

We write $[q]$ for “the units of the quantity q ”. Two quantities are *commensurable* when $[\cdot]$ agree (same exponent vector) and *incommensurable* otherwise.

The number of *base* units B is small and fixed (three, for mechanics). The number of *derived* units that might appear is unlimited: from M, L, T alone one can build L^2 , L/T , L/T^2 , ML , M/L^3 , $\dots$, endlessly. This gap — few base units, unboundedly many derived ones — is the whole point of the construction to come.

1.3 The demand, and why the obvious methods fail it

To (P1)–(P3) we add a fourth, physical, requirement:

(P4) Unit-honesty: every operation used to build f is dimensionally legal — we only ever *add* or *compare* quantities of the *same* unit, and combine different units only by *multiplying* (which is always legal: units just multiply too).

Remark 1.2 (why honesty is not optional). Suppose an encoder sorts all the entries of all the vectors into one long list by size. Sorting compares entries pairwise; as soon as it compares a mass against a length it has asked a meaningless question, and — worse — the answer flips if we re-express lengths in feet instead of metres. The output then jumps around under a mere change of units, in a wild, discontinuous way. Any method whose *decisions* (orderings, which-bucket-it-lands-in) depend on comparing incommensurable quantities is, in this sense, not measuring the physics; it is measuring our arbitrary choice of units. (P4) rules this out.

What is and isn't allowed. Picture each quantity as carrying a coloured tag (its unit). *Adding* or *comparing* is only allowed between same-coloured tags. *Multiplying* is always allowed — the product just gets a new colour (the product of the tags). Unit-honesty means: build the encoding using same-colour additions and free multiplications only, never a mixed-colour addition or comparison.

1.4 The plan

- Section 2 introduces the engine in the unit-free case: *encode by factorise*. A single vector becomes a polynomial; a multiset becomes a product of polynomials; the multiset is read back by factorising. This already gives (P1)–(P3), robustly.
- Section 3 puts the units back and locates exactly where the naive version sins against (P4) — and what it would cost to atone clumsily.
- Section 4 gives the fix: carry each entry's dimension on accessory variables, *one per base unit*. This restores (P4) at a price set by the number of *base* units.
- Section 6 works out that price and shows, with a worked check, that dimensionally uniform data costs no more than unit-free data.

Throughout, (P1), (P3), (P4) will be easy to see as we go; the one genuinely substantial fact is (P2), injectivity, and it rests on a single clean idea — *a product of distinct linear factors can be factorised in only one way*.

2 The engine: encode by factorise

We first ignore units entirely and build an encoding with properties (P1)–(P3). The idea is short: turn the multiset into a single polynomial whose *factors* are the vectors, and read the vectors back by factorising.

2.1 One vector becomes a polynomial

Introduce a formal *bookkeeping variable* t — a symbol we do arithmetic with but never assign a value to. Encode a vector $\mathbf{v} = (v_1, \dots, v_k)$ as the polynomial that uses its entries as coefficients:

$$p_{\mathbf{v}}(t) = v_1 + v_2 t + v_3 t^2 + \dots + v_k t^{k-1}. \quad (1)$$

This loses nothing: the entries *are* the coefficients, so $\mathbf{v}$ and $p_{\mathbf{v}}$ carry exactly the same information — read the coefficients off and you have the vector back. (A polynomial is determined by its coefficients, and vice versa.)

2.2 The multiset becomes a product

Introduce a second bookkeeping variable w , and encode the whole multiset as the product

$$F(w, t) = \prod_{j=1}^n (w - p_{\mathbf{v}^j}(t)). \quad (2)$$

Multiplied out, F is an ordinary polynomial in the two variables w and t ; **its list of coefficients is our encoding f** . Two of our properties are already visible:

- **(P1) Permutation-invariance.** A product does not care about the order of its factors, so reordering the vectors $\mathbf{v}^j$ leaves F — and hence every coefficient — unchanged.
- **(P3) Continuity.** Each coefficient of F is built from the v_i^j by additions and multiplications only (no division, no sorting), so it is a polynomial in the entries, hence continuous — indeed smooth.

2.3 Reading it back: factorisation

Everything rests on recovering the multiset from the coefficients of F . Here is the key fact, stated plainly.

A polynomial factorises in only one way. Just as a whole number splits into primes in only one way ($12 = 2 \cdot 2 \cdot 3$ and nothing else), a polynomial in w splits into linear factors $(w - r)$ in only one way: the r 's are its roots, and they are what they are. So from F we can recover the multiset of factors $\{w - p_{\mathbf{v}^j}(t)\}$, hence the multiset of polynomials $\{p_{\mathbf{v}^j}(t)\}$, hence (reading coefficients) the multiset of vectors. The order is lost in the product — which is exactly (P1) — but the multiset is not.

Theorem 2.1 (injectivity). *The encoding f (the coefficients of F) determines the multiset $\{\mathbf{v}^1, \dots, \mathbf{v}^n\}$ completely. Hence (with (P1), (P3) above) f is a continuous, permutation-invariant embedding.*

Proof. Knowing all coefficients of F means knowing the polynomial $F(w, t)$. View it as a polynomial in w whose coefficients happen to be polynomials in t . Such a polynomial factors uniquely into monic linear pieces $w - q(t)$; comparing with (2), the multiset of pieces is exactly $\{w - p_{\mathbf{v}^j}(t)\}$, so the multiset $\{p_{\mathbf{v}^j}(t)\}$ is determined. Each $p_{\mathbf{v}^j}$ determines $\mathbf{v}^j$ by reading off coefficients (1). So the whole multiset is recovered. This holds for *every* input, with no exceptions: if two vectors are equal the product simply has a repeated factor, and the repeat is faithfully recorded. $\square$

Remark 2.2 (robust, always valid). Notice what did *not* happen: no division (so nothing can blow up), no sorting (so no kinks), no genericity assumption (it works for *all* inputs, including coincidences). This robustness will survive into the unit-honest version.

2.4 A worked example

Example 2.3 (encode and recover). Take $n = 2$ vectors in $k = 2$ dimensions,

$$\mathbf{v}^1 = (3, 5), \quad \mathbf{v}^2 = (2, 1),$$

so $p_1(t) = 3 + 5t$ and $p_2(t) = 2 + t$. Multiply out (2):

$$F(w, t) = (w - (3 + 5t))(w - (2 + t)) = w^2 - (5 + 6t)w + (6 + 13t + 5t^2).$$

The encoding is the coefficient list

$$f = \left(\underbrace{1}_{w^2}, \underbrace{-5, -6}_{w^1: t^0, t^1}, \underbrace{6, 13, 5}_{w^0: t^0, t^1, t^2} \right).$$

Now recover, pretending we were only handed f . Reassemble $F = w^2 - (5 + 6t)w + (6 + 13t + 5t^2)$ and solve for w (the quadratic formula, with t carried along):

$$w = \frac{(5 + 6t) \pm \sqrt{(5 + 6t)^2 - 4(6 + 13t + 5t^2)}}{2} = \frac{(5 + 6t) \pm \sqrt{1 + 8t + 16t^2}}{2} = \frac{(5 + 6t) \pm (1 + 4t)}{2},$$

because $1 + 8t + 16t^2 = (1 + 4t)^2$. The two roots are

$$w = 3 + 5t \quad \text{and} \quad w = 2 + t,$$

i.e. exactly $p_1(t)$ and $p_2(t)$. Reading their coefficients returns the multiset $\{(3, 5), (2, 1)\}$ — and note we recovered it as a *set*: nothing in f remembers which vector we called “first”. (Had we started from the relabelled list $\{(2, 1), (3, 5)\}$ we would have multiplied the same two factors in the other order and obtained the identical F .)

For larger n the polynomial in w has degree n and one factorises a degree- n polynomial instead of a quadratic, but the principle is identical: the coefficients are the encoding, and unique factorisation hands the multiset back.

3 Putting the units back: where the engine sins

The engine of Section 2 is clean, robust, and cheap. But it was built for plain numbers, and the moment the entries carry units it quietly breaks rule (P4).

3.1 The hidden illegal addition

Look again at the per-vector polynomial (1):

$$p_{\mathbf{v}}(t) = v_1 + v_2 t + v_3 t^2 + \dots$$

For this *sum* to mean anything, all its terms must share a unit: $[v_1] = [v_2 t] = [v_3 t^2] = \dots$. The first equality forces $[t] = [v_1]/[v_2]$; feeding that into the second forces $[v_3] = [v_2]^2/[v_1]$; the next forces $[v_4] = [v_2]^3/[v_1]^2$; and so on. In words, the construction is only legal if the units of the entries form a rigid *geometric progression* — which real data has no reason to do. Generically, $p_{\mathbf{v}}$ adds a mass to a length, and **that is the sin**.

Example 3.1 (the sin, and the metre/foot wildness). Let each vector be (mass, length), e.g. $\mathbf{v} = (2 \text{ kg}, 3 \text{ m})$. Then $p_{\mathbf{v}}(t) = 2 \text{ kg} + 3 \text{ m} \cdot t$ adds a mass to a length times t . This is meaningful only if t secretly carries the units kg/m — a contrived choice — and even then, a vector with a *third* entry of some unrelated unit (a time, say) could not be rescued by the same t . Worse, the coefficient “ $2 + 3t$ ” mixes the two numbers 2 and 3, whose *ratio* is an accident of our units: in feet the length 3m becomes 9.84ft, so the same physical vector now reads $2 + 9.84t$, and every downstream coefficient of F shifts in a way that has nothing to do with the physics. The encoding is reporting our choice of ruler.

3.2 The clumsy cure, and why it is too expensive

There is an obvious repair: never put two different units into the same sum. Give each *distinct unit* its own bookkeeping variable, so that every addition is between like and like. For $\mathbf{v} = (\text{mass}, \text{length})$ one would write something like $z_M^? \dots$ with one variable z_M “carrying mass” and another z_L “carrying length”, keeping the two entries on separate variables and only ever multiplying across them. This *is* unit-honest. The trouble is the bookkeeping cost:

One variable per unit is unaffordable. If we spend one accessory variable on each distinct unit that appears, the number of variables grows with the number of distinct *derived* units in the data — and those are unbounded: L^2 , L/T , L/T^2 , ML , M/L^3 , $\dots$. A dataset mixing forces, energies, densities and velocities could need a dozen variables, each inflating the size of the encoding. We would be paying for the *zoo of derived units*, when the physics is built from only $B = 3$ base ones.

So we are caught between a cheap-but-dishonest encoder (Section 2, one variable t , but it adds incommensurables) and an honest-but-bloated one (one variable per derived unit). The next section escapes the dilemma by spending variables on *base* units only — of which there are just B — and letting *powers* of them name every derived unit that arises.

4 The fix: carry dimensions on base variables

4.1 Base accessory variables, and the homogenising monomial

Introduce one bookkeeping variable per *base* unit: $\xi = (\xi_1, \dots, \xi_B)$, where ξ_b is declared to carry base unit b (so ξ_1 “is a mass”, ξ_2 “is a length”, ξ_3 “is a time”). For a unit with exponent vector $\mathbf{d} = (d_1, \dots, d_B)$ write the monomial

$$\xi^{\mathbf{d}} = \xi_1^{d_1} \xi_2^{d_2} \dots \xi_B^{d_B}, \quad \text{which carries exactly the unit } [\xi^{\mathbf{d}}] = \prod_b (\text{base } b)^{d_b}.$$

The key move is one line. Entry v_i of a vector has some fixed unit, with exponent vector $\mathbf{d}_i$. Then $\xi^{\mathbf{d}_i}$ has the *same* unit as v_i , so the ratio

$$v_i \xi^{-\mathbf{d}_i} \quad \text{is dimensionless:} \quad [v_i \xi^{-\mathbf{d}_i}] = [v_i] / [\xi^{\mathbf{d}_i}] = 1. \quad (3)$$

We have “paid off” the dimension of v_i using powers of the base variables. Since only B base variables are ever used, a derived unit no matter how exotic (ML^2/T^2 , M/L^3 , $\dots$) is just *some power-pattern* of the same B variables — never a new variable.

4.2 The unit-honest per-vector encoder

Now repeat the engine of Section 2, but homogenise every entry first. Keep the tag variable t to tell entries apart, and define

$$\Psi(t, \xi; \mathbf{v}) = \sum_{i=1}^k v_i t^{i-1} \xi^{-\mathbf{d}_i}. \quad (4)$$

Every term is dimensionless by (3), so this is a sum of like with like: **no illegal addition anywhere** — (P4) holds. The two “carrier” factors do two different jobs:

- $\xi^{-\mathbf{d}_i}$ *pays the dimensional debt* of v_i and, as a bonus, routes entries of different units onto different ξ -monomials;
- t^{i-1} *tags* the entries, so that even two entries of the *same* unit (which share a ξ -monomial) land on different t -powers and stay distinguishable.

Together they make the k “addresses” $t^{i-1} \xi^{-\mathbf{d}_i}$ all distinct, so the coefficient sitting at address i is exactly v_i : Ψ *loses nothing*, just as $p_{\mathbf{v}}$ did, but now legally.

4.3 The multiset encoder

Form the product over the vectors, against a fresh dimensionless variable w :

$$\Phi(w, t, \xi) = \prod_{j=1}^n (w - \Psi(t, \xi; \mathbf{v}^j)). \quad (5)$$

(Subtracting the dimensionless Ψ from the dimensionless w is legal.) The encoding f is, as before, the list of coefficients of Φ — now a polynomial in the $B + 2$ variables $w, t, \xi_1, \dots, \xi_B$. Recovery is the same two-step factorisation: factor Φ in w to get the multiset $\{\Psi^j\}$; read each Ψ^j off its distinct addresses to get $\mathbf{v}^j$. We collect the guarantees in Section 5; first, a worked example.

4.4 A worked example with real units

Example 4.1 (mass and two lengths). Let each vector hold a mass and two lengths, e.g. a particle’s mass and its (x, y) position. The base units in play are M and L (so here $B = 2$; order (M, L)), and the entries have exponent vectors

$$\mathbf{d}_1 = (1, 0) \text{ [mass]}, \quad \mathbf{d}_2 = (0, 1) \text{ [length]}, \quad \mathbf{d}_3 = (0, 1) \text{ [length]}.$$

Take the two-vector multiset

$$\mathbf{v}^1 = (2, 3, 5), \quad \mathbf{v}^2 = (4, 1, 6)$$

(masses 2, 4; positions (3, 5), (1, 6)). Homogenise each entry by (4), using $\xi^{-\mathbf{d}_1} = \xi_M^{-1}$ and $\xi^{-\mathbf{d}_2} = \xi^{-\mathbf{d}_3} = \xi_L^{-1}$:

$$\Psi^1 = \underbrace{2\xi_M^{-1}}_{t^0} + \underbrace{3t\xi_L^{-1}}_{t^1} + \underbrace{5t^2\xi_L^{-1}}_{t^2} = \frac{2}{\xi_M} + \frac{3t + 5t^2}{\xi_L}, \quad \Psi^2 = \frac{4}{\xi_M} + \frac{t + 6t^2}{\xi_L}.$$

Check the honesty: in Ψ^1 the mass term $2/\xi_M$ has unit $M/M = 1$, and the length terms $3t/\xi_L$, $5t^2/\xi_L$ have unit $L/L = 1$ — all three dimensionless, so adding them is legal. Note how the two lengths share the ξ_L “channel” but are kept apart by t versus t^2 , while the mass sits on its own ξ_M channel.

The multiset encoding is the coefficient list of $\Phi = (w - \Psi^1)(w - \Psi^2) = w^2 - (\Psi^1 + \Psi^2)w + \Psi^1\Psi^2$, where

$$\Psi^1 + \Psi^2 = \frac{6}{\xi_M} + \frac{4t + 11t^2}{\xi_L},$$

and $\Psi^1\Psi^2$ expands into terms on the monomials ξ_M^{-2} , $\xi_M^{-1}\xi_L^{-1}$, ξ_L^{-2} (times powers of t). For instance the $\xi_M^{-1}\xi_L^{-1}$ part of $\Psi^1\Psi^2$ is

$$2(t + 6t^2) + 4(3t + 5t^2) = 14t + 32t^2,$$

and the ξ_M^{-2} part is $2 \cdot 4 = 8$. Every coefficient is a plain number (with a definite unit attached, namely the inverse of its ξ -monomial) obtained *without a single illegal addition*.

Recovery runs as in Example 2.3: factor Φ in w (it is a quadratic in w whose two roots, carried along in t, ξ , are Ψ^1 and Ψ^2); then in each root read the coefficient of ξ_M^{-1} (the mass), of $t\xi_L^{-1}$ and of $t^2\xi_L^{-1}$ (the two lengths). Out come $\{(2, 3, 5), (4, 1, 6)\}$, as a multiset.

Remark 4.2 (the metre/foot wildness is gone). Re-run Example 3.1’s worry. Switch lengths from metres to feet: every length number scales by $\lambda = 3.28$, but ξ_L “is a length” and scales the same way, so each ratio $v_i\xi_L^{-1}$ is *unchanged*. The construction is blind to the choice of ruler. More precisely, a coefficient of Φ that sits on the monomial $\xi^{\mathbf{m}}$ carries the definite unit $\xi^{\mathbf{m}}$, and when you change units it simply rescales by that one honest factor — never the wild reshuffle of Example 3.1. (You cannot make the output a pure number with no units attached; that would need an arbitrary external scale, which is exactly what we refuse. What you get instead is an output whose every component is correctly *labelled* with its own unit.)

5 Why it works

All four properties now fall out, and the same robustness as the unit-free engine comes with them.

Theorem 5.1 (the homogenised encoder is a unit-honest embedding). *The coefficient list of Φ in (5) is a map of multisets that is permutation-invariant (P1), injective (P2), continuous (P3) and unit-honest (P4); and it is defined, and injective, for every input, with no genericity assumption.*

Proof. (P1) Φ is a product over the vectors, so reordering them leaves it unchanged.

(P3) Each coefficient of Φ is built from the entries v_i^j by additions and multiplications only — the ξ - and t -powers are fixed bookkeeping — so it is a polynomial in the entries, hence continuous.

(P4) Every Ψ^j is a sum of dimensionless terms (3), and Φ subtracts and multiplies dimensionless quantities; the only place units meet is inside the monomials $\xi^{\mathbf{m}}$, where they *multiply*. There is no addition or comparison of unlike units anywhere.

(P2), and *always-valid*. Knowing the coefficients is knowing Φ as a polynomial in w (with coefficients that are Laurent polynomials in t, ξ). It factors uniquely into monic linear pieces $w - \Psi$, so the multiset $\{\Psi^j\}$ is recovered. Within each $\Psi^j = \sum_i v_i^j t^{i-1} \xi^{-\mathbf{d}_i}$ the addresses $t^{i-1} \xi^{-\mathbf{d}_i}$ are distinct (distinct t -powers), so the coefficient at address i is exactly v_i^j . Thus every $\mathbf{v}^j$ is recovered, and the whole multiset with it. No step divides or assumes genericity: equal vectors give a repeated factor, recorded faithfully. $\square$

Remark 5.2 (do we even need t ?). The tag t only earns its keep when two entries of one vector share a unit (then they share a ξ -monomial and t is what separates them — the two lengths of Example 4.1). If *every* entry has a distinct unit, the ξ -monomials already separate them and one may drop t entirely. This small economy matters for the count, next.

6 What it costs

The encoding is the coefficient list of Φ , so its size is the number of distinct monomials $w^a t^b \xi_1^{c_1} \cdots \xi_B^{c_B}$ that appear. Two things are already clear and reassuring:

- **The number of variables is $B + 2$** (w, t , and the B base variables) — fixed, and *independent of how many derived units appear*. The zoo of derived units of Section 3 has been tamed: each is a power-pattern of the same B variables.
- The degrees are modest: $\leq n$ in w , $\leq n(k - 1)$ in t , and $\leq nA_b$ in each ξ_b , where A_b is the spread of base- b exponents across the entries (how far apart the powers of base unit b run).

6.1 Counting honestly: not every box is filled

One might fear the worst case “degree in each variable multiplied together”, i.e. $n \cdot n(k - 1) \cdot \prod_b nA_b = O(n^{B+2})$. But that badly overcounts, because the exponents are *not free to vary independently*. The ξ -power of any term of Φ is fixed by *which* entries went into it, which also fixes the w - and t -powers; the three move together, not in an independent grid. The honest count is the number of lattice points actually swept out — which grows like n^d , where d is the number of *independent directions* the exponents can move:

$$d = 1 + \text{rank}(R), \quad (6)$$

where R is a small table of plain numbers, built as follows. Pick component 0 as a reference. Then for each *other* component $c = 1, \dots, k - 1$ write one row of R : the index c itself, followed by the B differences $\mathbf{d}_c - \mathbf{d}_0$ between that component’s unit-exponents and component 0’s. So R has $k - 1$ rows and $1 + B$ columns, and $\text{rank}(R)$ is its *ordinary matrix rank* — the number of linearly independent rows (equivalently, of columns). Then

$$\#\text{coefficients} = O(n^d \cdot C), \quad 2 \leq d \leq B + 2,$$

with C a constant absorbing the small extents (k , and the exponent spreads).

It pays to see what each column of R contributes. The first column — the indices $1, 2, \dots, k-1$ — is never all zero, so it always supplies one independent direction; that is the unavoidable “+1” which, with the w of (5), gives the base cost n^2 . The other B columns are the *unit-difference* columns, and they raise the rank only when the units *actually vary*. **If every entry carries one common unit, those columns are identically zero** and add nothing: $\text{rank}(R) = 1$ and $d = 2$. A derived unit, however exotic, costs extra only insofar as it makes these B columns span a new direction — and there are only B of them. **So the cost is set by how the units *vary* across the B base axes (capped at B), never by the sheer number of distinct derived units on display.**

6.2 Sanity check: uniform units cost nothing extra

The formula (6) makes a prediction worth checking by hand: if every entry carries the *same* unit, the dimensions should be “free” and the cost should drop to that of the unit-free engine, $O(n^2k)$. They do.

Example 6.1 (a multiset of positions). Let each vector be a position $\mathbf{v} = (x, y, z)$, all three entries lengths ($k = 3$, all unit L). Because the units are identical, the unit-difference columns of the table R are entirely zero, leaving $\text{rank}(R) = 1$, so (6) predicts $d = 2$. We confirm it by direct count. Every $\xi^{-\mathbf{d}_i}$ is the same monomial ξ_L^{-1} , so it factors out of Ψ as a single prefactor:

$$\Psi(t, \xi_L; \mathbf{v}) = \xi_L^{-1} (x + y t + z t^2) = \xi_L^{-1} p_{\mathbf{v}}(t).$$

Then

$$\Phi = \prod_{j=1}^n (w - \xi_L^{-1} p_{\mathbf{v}^j}(t)) = \sum_{i=0}^n (-1)^i \xi_L^{-i} e_i(p_{\mathbf{v}^1}, \dots, p_{\mathbf{v}^n}) w^{n-i},$$

where e_i is the sum of all products of i of the $p_{\mathbf{v}^j}$. Look at the ξ_L -powers: the coefficient of w^{n-i} *always* carries ξ_L^{-i} . The power of ξ_L is welded to the power of w ; it is not a free direction at all. So the genuinely distinct monomials are just $w^{n-i} t^q$ (the ξ_L tagging along), and counting them,

$$\#\text{coefficients} = \sum_{i=0}^n (\deg_t e_i + 1) = \sum_{i=0}^n (i(k-1) + 1) = O(n^2k),$$

exactly the unit-free cost — the $d = 2$ that $\text{rank}(R) = 1$ predicted. The ξ contributes no independent direction: with one common unit the unit-difference columns of R vanish, and the lone ξ_L rides along on w for free.

6.3 The dial

So the whole price is one matrix rank. A few cases, with R built and its rank read straight off:

data	$\text{rank}(R)$	d	cost
all one unit (e.g. positions, any k)	1	2	$O(n^2k)$ — as cheap as unit-free
any $k = 2$ (so R has just one row)	1	2	$O(n^2k)$
(mass, length, length), $k = 3$	2	3	$O(n^3k)$
units spanning all B axes, one shared	$B+1$	$B+2$	$O(n^{B+2})$

(The third row is the running example of Section 4: its two lengths share a unit, so the index column is independent of the single mass-versus-length difference, giving $\text{rank}(R) = 2$.) The point of the construction is the cap: a dataset mixing a dozen *derived* units (velocities, forces, energies, densities, ...) built from $B = 3$ base ones still has $\text{rank}(R) \leq B + 1 = 4$, so $d \leq 5$ and the cost is $O(n^{\leq 5})$ — no matter how many derived units appear. They are free riders on the three base axes.

7 Summary

We wanted to encode a multiset of n vectors whose entries carry physical units, keeping the usual virtues — permutation-invariance, injectivity, continuity — while never adding or comparing two quantities of different units. The method:

1. **Encode by factorise** (Section 2). Turn each vector into a polynomial $p_{\mathbf{v}}(t)$, the multiset into the single product $F = \prod_j (w - p_{\mathbf{v}^j})$, and recover it by unique factorisation. This is automatically invariant, continuous and injective, with no division and no genericity.
2. **Homogenise on base variables** (Section 4). The naive $p_{\mathbf{v}}$ secretly adds incommensurable entries (Section 3). Cure it by attaching to each entry the monomial $\xi^{-\mathbf{d}_i}$ in B base-unit variables, making every term dimensionless: $\Psi = \sum_i v_i t^{i-1} \xi^{-\mathbf{d}_i}$, $\Phi = \prod_j (w - \Psi^j)$. Now every operation is unit-legal, and a change of units merely rescales each output by its own honest factor.
3. **Pay only for base units** (Section 6). The encoding uses $B + 2$ variables regardless of how many derived units appear, and its size is $O(n^d)$ with $d = 1 + \text{rank}(R) \leq B + 2$ — the rank measuring how the units *vary* across the B base axes, and dropping to the unit-free $O(n^2k)$ ($d = 2$) whenever the entries are dimensionally uniform.

The single idea underneath is worth restating: *a derived unit is just a power-pattern of the base units, so B accessory variables can name them all.* That is what converts the unbounded zoo of derived dimensions into a fixed, small overhead — and lets a method that would otherwise grow with the imagination of the unit-system grow instead only with the three pillars M, L, T that the physics is actually built from.

Remark 7.1 (relation to comparison-based encoders). A different family of encoders (sorting- or subdivision-based) can reach a smaller output, of size $O(nk)$, but only by *comparing* entries by magnitude — and across different units that comparison is exactly the illegal operation (P4) forbids. The trade is real: comparison buys a smaller code at the cost of unit-honesty; the factorise-and-homogenise method here buys unit-honesty at the cost of a larger, but still polynomial, code whose exponent is set by how the units vary across the B base axes. One pays in magnitude-comparisons, the other in the extra powers of n counted by $\text{rank}(R)$; which is the better bargain depends on whether the units are genuinely incommensurable.

A Implementation

The method of this note is implemented, and tested, in the `Echinocoder` repository (run under its `venv`). The class is `UnitAwareBursarialEncoder` (also exposed as `Embedder`, the repository’s standard name) in

https://github.com/kesterlester/Echinocoder/blob/main/Cinf_sympy_unitAwareBursar_embedder_for_array_of_reals_as_multiset.py

It is a `MultisetEmbedder` and uses `sympy` for the polynomial algebra. It is the unit-aware sibling of the repository’s existing (dimensionless) Bursarial encoders — the “not-fair” `Cinf_sympy_bursar...` of Section 2 and the “fair” `Cinf_sympy_evenBursar...`.

What corresponds to what

This note	Code
the units, $\mathbf{d}_i$ (one per entry)	<code>dimensions</code> , the list of k integer exponent vectors passed to the constructor; this fixes k and B
homogenised per-vector Ψ (4)	<code>_psi</code> ; the per-entry address $t^{i-1}\xi^{-\mathbf{d}_i}$ is <code>_address</code>
multiset polynomial Φ (5)	<code>_phi_poly</code> , an expanded <code>sympy.Poly</code> in the gens $w, t, \xi_0, \dots, \xi_{B-1}$
the encoding f (coefficients of Φ)	<code>embed_generic</code> ; the coefficients are read over a fixed support (<code>_canonical_support</code>)
the encoding size $O(n^{1+\text{rank}(R)})$	<code>size_from_n_k</code> ; returns -1 for any k other than the one fixed at construction

Three implementation remarks

- **Non-negative exponents.** The code uses $\xi^{\mathbf{D}-\mathbf{d}_i}$ in place of $\xi^{-\mathbf{d}_i}$, where $\mathbf{D}$ is the entry-wise maximum of the $\mathbf{d}_i$. As noted in the body, this is one legal multiplication by the fixed monomial $\xi^{\mathbf{D}}$: it gives every term the common unit base $^{\mathbf{D}}$ instead of being dimensionless, so the additions remain honest, and it keeps all ξ -exponents ≥ 0 so ordinary (non-Laurent) polynomials suffice. Injectivity, invariance and continuity are untouched.
- **Fixed support, so the size is predictable.** The output is the coefficient of each monomial in a *data-independent* support — the monomials Φ carries for generic data, computed once from a symbolic template and cached, with the trivial leading w^n term (always 1) dropped. Coefficients that happen to vanish for particular data are still emitted (as 0). This is why the embedding length always equals `size_from_n_k` exactly, as the framework asserts.
- **What the test checks.** The accompanying test confirms, on worked inputs: permutation-invariance; injectivity (distinct multisets give distinct codes, relabelled ones give identical codes, repeated-vector inputs encode cleanly); the size matches `size_from_n_k` and the latter returns -1 for the wrong k ; the dimensional-covariance law — each coefficient rescales by $\lambda^{(n-a)D_b-m_b}$ under a change of base unit b by λ (Section 4); derived units with negative exponents (velocity, energy) work; and the all-positions case collapses to the unit-free Bursar size $n + (k-1)n(n+1)/2$, confirming the $d = 2$ (i.e. $\text{rank}(R) = 1$) prediction of Section 6.